PR-02

Aprendendo a Programar em Python para Saúde Digital



Primeiros programas em Python no navegador, com exemplos de saúde digital e sem exigir experiência anterior.

3 h | sem pré-requisitos | Google Colab

Prof. Dr. Guillermo Alvarez Bestard

Projeto Ocean · Residência em TIC 53 - Brasília

Você pode começar por aqui

O curso foi desenhado para quem nunca programou em Python.

NÃO É NECESSÁRIO TER FEITO O PR-01

Se você participou do curso com Scratch, algumas ideias parecerão familiares.
Se não participou, nada ficará faltando.

Os conceitos essenciais de algoritmo, variável, decisão e repetição serão apresentados novamente antes de aparecerem em código.

COMEÇAMOS DO ZERO

O objetivo é aprender a pensar e escrever pequenos programas — não decorar sintaxe.

Onde este curso está na trilha

PR-02 é independente, mas prepara o terreno para dados e monitoramento.

OPCIONAL

PR-01

Lógica com Scratch

HOJE

PR-02

Python para Saúde

PRÓXIMO

PR-03

Dados com Python + Pandas

APLICAÇÃO

PR-04

Monitoramento em tempo real

O PR-02 entrega a base textual: dados → regras → repetição → funções → registros.

O que você será capaz de fazer hoje

01

Executar Python

Criar células no Colab, rodar código e interpretar saídas e erros básicos.

02

Trabalhar com dados

Usar variáveis, tipos, entrada, conversões e operadores.

03

Decidir e repetir

Aplicar if/elif/else, listas, for e compreender while.

04

Organizar o código

Criar funções pequenas e representar leituras com dicionários.

05

Testar e explicar

Usar casos de fronteira, localizar erros e justificar a lógica.

Meta: terminar com um notebook que executa do início ao fim e resolve um pequeno problema aplicado.

Como vamos usar as 3 horas

Explicação curta → demonstração → prática → integração.



Mais tempo será gasto escrevendo e testando código do que ouvindo explicações.

Projeto condutor: um wearable simulado

1 · ENTRADA

uma leitura sintética de 0 a 100

2 · REGRA

classificar em A / B / C

3 · REPETIÇÃO

processar várias leituras

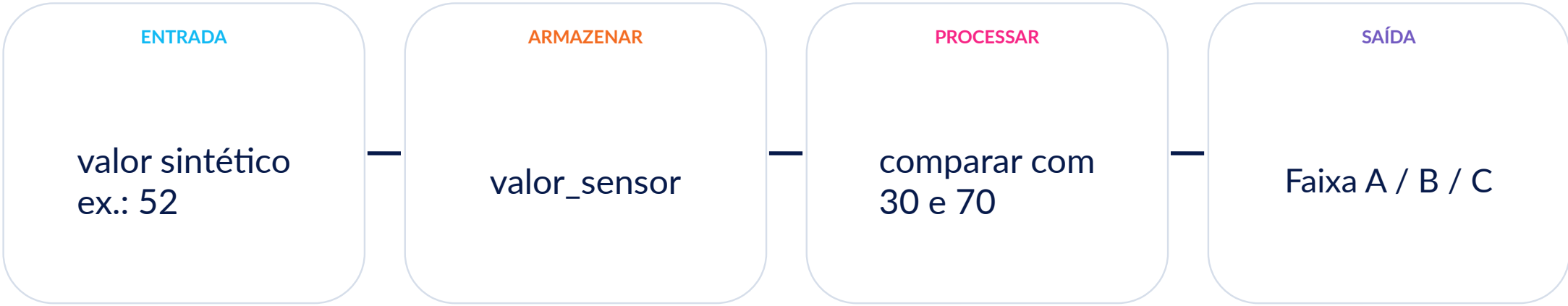
4 · REGISTRO

guardar valor, estado e qualidade

Usaremos o mesmo problema durante todo o encontro, construindo o programa por etapas.

Todo programa tem um fluxo

Antes da sintaxe, enxergue entrada → processamento → saída.



O mesmo raciocínio vale para sensores, formulários, APIs, arquivos ou qualquer outra fonte de dados.

As faixas 30 e 70 existem apenas para ensinar programação. Não têm significado clínico.

Algoritmo antes do código

Escreva a ideia em passos claros. Depois transforme os passos em Python.

PSEUDOCÓDIGO

```
1. receber um valor
2. guardar o valor
3. se valor < 30 → A
4. senão, se valor ≤ 70 → B
5. senão → C
6. mostrar o resultado
```

Pergunta para a turma

1) Em qual passo o programa precisa "tomar uma decisão"?

2) E qual passo precisará se repetir quando houver várias leituras?

Google Colab: Python no navegador

Não precisamos instalar Python para acompanhar a aula.



- Cada célula pode conter código ou texto.
- ▶️ executa a célula atual.
- A saída aparece logo abaixo.
- A ordem de execução importa.

colab.research.google.com

Primeiro programa

print() envia uma mensagem para a saída.

CÉLULA PYTHON

```
print("Olá, saúde digital!")
print(2 + 3)
```

SAÍDA

```
Olá, saúde digital!
5
```

Experimento rápido

- Troque a mensagem.
- Mude 2 + 3 para outra expressão.
- Execute novamente.

Código → execução → saída

Comentários e erros fazem parte

Ler a mensagem de erro é uma habilidade de programação.

EXEMPLO COM ERRO

```
# isto é um comentário
print("Leitura recebida")
print(valor_inexistente)
```

SAÍDA

```
NameError: name 'valor_inexistente' is not defined
```

1

Leia a última linha do erro

2

Localize a linha indicada

3

Corrija uma coisa por vez

Variáveis: nomes para valores

Uma variável guarda um valor que pode ser consultado ou substituído.



ATRIBUIÇÃO

```
valor_sensor = 52
print(valor_sensor)

valor_sensor = 78
print(valor_sensor)
```

SAÍDA

```
52
78
```

O nome permanece. O valor pode mudar durante a execução.

Tipos básicos

O tipo do valor influencia as operações que fazem sentido.

INT

52
número inteiro

FLOAT

52.5
número decimal

STR

"OK"
texto

BOOL

True / False
lógico

```
valor = 52.5
print(type(valor))
```

SAÍDA

```
<class 'float'>
```

type() e conversão

Às vezes precisamos transformar um valor antes de usá-lo.

CONVERSÃO DE TEXTO PARA NÚMERO

```
texto = "52.5"
print(type(texto))

valor = float(texto)
print(type(valor))
```

SAÍDA

```
<class 'str'>
<class 'float'>
```

Conversões frequentes

```
int("52")   → 52
float("52.5") → 52.5
str(52)     → "52"
bool(...)   → True / False
```

Converter não “corrige” o dado. Apenas muda sua representação.

input(): recebendo dados do usuário

input() sempre devolve texto — mesmo quando digitamos um número.

ENTRADA + CONVERSÃO

```
valor_texto = input("Digite um valor de 0 a 100: ")
print(type(valor_texto))

valor_sensor = float(valor_texto)
print(valor_sensor)
```

PASSO 1

receber texto

PASSO 2

converter para float

SAÍDA

```
<class 'str'>
52.0
```

Pergunta: o que aconteceria se tentássemos comparar o texto diretamente com 30?

CHECKPOINT · Receber e mostrar uma leitura

Tarefa de 3 minutos

1. Peça ao usuário uma leitura sintética.
2. Converta para float.
3. Mostre: “Leitura recebida: X”.

SUA CÉLULA

```
# escreva aqui
```

Teste pelo menos com 30 e 70.

Operadores aritméticos

Expressões produzem novos valores.

+	-	*	/
somar	subtrair	multiplicar	dividir

```
valor = 52
ajuste = 10
resultado = valor + ajuste
print(resultado)
```

SAÍDA

```
62
```

Operadores relacionais

Comparações produzem True ou False.

```
52 < 30
```

False

```
52 <= 70
```

True

```
52 == 52
```

True

```
52 != 70
```

True

== compara valores.

= atribui um valor a uma variável.

Operadores lógicos

and, or e not combinam ou invertem condições.

AND

as duas condições precisam ser verdadeiras

OR

basta uma condição ser verdadeira

NOT

inverte True ↔ False

```
valor = 52
qualidade_ok = True

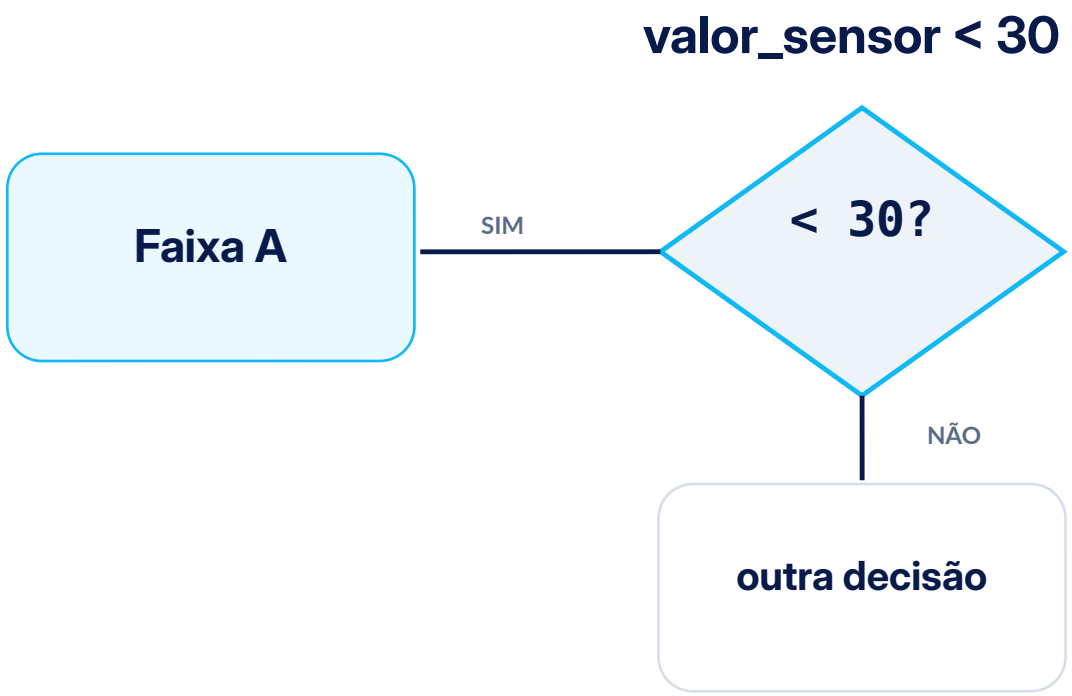
print(valor <= 70 and qualidade_ok)
print(valor < 30 or valor > 70)
print(not qualidade_ok)
```

SAÍDA

```
True
False
False
```

Programas precisam decidir

Uma condição escolhe qual caminho será executado.



if / elif / else

Três faixas didáticas com uma única cadeia de decisão.

CLASSIFICAÇÃO

```
valor_sensor = 52

if valor_sensor < 30:
    estado = "Faixa A"
elif valor_sensor <= 70:
    estado = "Faixa B"
else:
    estado = "Faixa C"

print(estado)
```

SAÍDA

```
Faixa B
```

IF

primeira pergunta

ELIF

outra pergunta, se necessário

ELSE

caso restante

Indentação faz parte da sintaxe

Em Python, os espaços dizem quais linhas pertencem a cada bloco.

CORRETO

```
if valor_sensor < 30:
    estado = "A"
    print("abaixo do limite")

print("fim")
```

ERRO DE INDENTAÇÃO

```
if valor_sensor < 30:
estado = "A"
print("abaixo do limite")
```

4 espaços é uma convenção comum

Origem na tipografia: Antigamente, quando os livros eram impressos com tipos de chumbo, a primeira linha de um parágrafo era deslocada para dentro. Historicamente, documentos oficiais também podiam ser recortados de forma ondulada ou "dentada" no topo para provar autenticidade.

Vem do latim ***dens*** (que significa "dente"), passando pelo verbo latino ***indentare*** ("colocar dentes" ou "recortar em forma de dentes"), e chegou ao português através do inglês ***indentation*** no contexto da ciência da computação.

Testar as fronteiras

Os casos próximos aos limites encontram erros rapidamente.

Entrada	Esperado	Por quê?
29	A	abaixo do primeiro limite
30	B	no primeiro limite
70	B	no segundo limite
71	C	acima do segundo limite

Fronteiras são mais informativas do que testar apenas valores “no meio” das faixas.

Listas: várias leituras em uma coleção

Uma lista mantém valores em ordem e permite acessá-los por posição.

LISTA

```
leituras = [22, 30, 45, 70, 90]

print(leituras)
print(leituras[0])
print(len(leituras))
```

SAÍDA

```
[22, 30, 45, 70, 90]
22
5
```

Índices

posição:	0	1	2	3	4
valor:	22	30	45	70	90

Em Python, a primeira posição é 0.

for: faça algo para cada item

O laço percorre a lista sem copiarmos o mesmo código muitas vezes.

PERCORRENDO A LISTA

```
leituras = [22, 30, 45, 70, 90]

for valor in leituras:
    print("Leitura:", valor)
```

SAÍDA

```
Leitura: 22
Leitura: 30
Leitura: 45
Leitura: 70
Leitura: 90
```

A cada repetição

valor recebe o próximo item da lista.

Quando a lista termina, o for termina.

Repetir a lógica completa

Cada nova leitura precisa ser classificada — não apenas impressa.

DECISÃO DENTRO DA REPETIÇÃO

```
leituras = [22, 30, 45, 70, 90]

for valor in leituras:
    if valor < 30:
        estado = "A"
    elif valor <= 70:
        estado = "B"
    else:
        estado = "C"
    print(valor, estado)
```

SAÍDA

```
22 A
30 B
45 B
70 B
90 C
```

Qual trecho precisa ficar indentado dentro do for?

TODO O PROCESSAMENTO DA LEITURA

Esse erro — deixar a classificação fora do laço — é muito comum.

while: repetir enquanto uma condição for verdadeira

Use quando a quantidade de repetições depende de uma condição.

REPETIÇÃO CONDICIONAL

```
contador = 0

while contador < 3:
    print("Tentativa", contador + 1)
    contador += 1

print("fim")
```

SAÍDA

```
Tentativa 1
Tentativa 2
Tentativa 3
fim
```

FOR

bom quando percorremos uma coleção

WHILE

bom quando repetimos até algo mudar

Cuidado: se a condição nunca mudar, o while pode não terminar.

Gerando leituras sintéticas

Um módulo pode fornecer funções que não fazem parte do núcleo básico.

VALORES ALEATÓRIOS

```
from random import randint

for _ in range(5):
    valor_sensor = randint(0, 100)
    print(valor_sensor)
```

SAÍDA

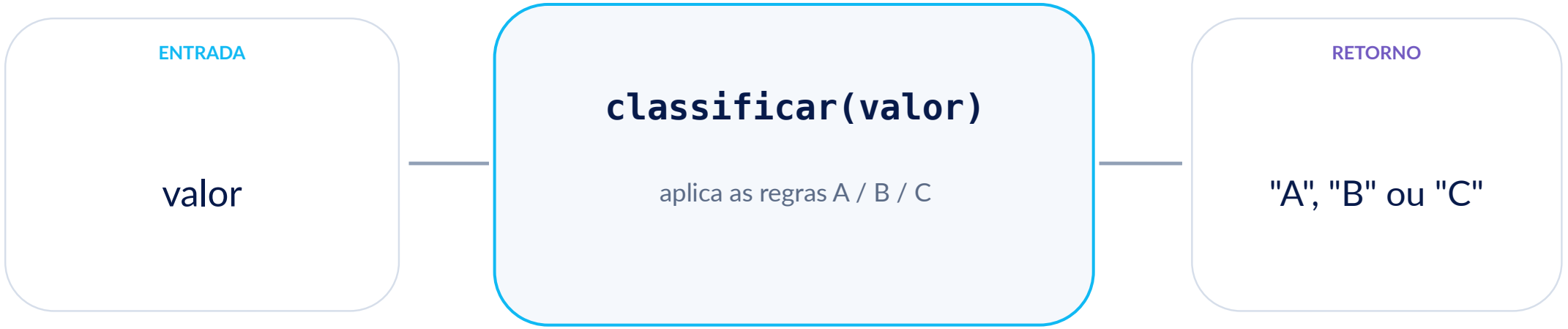
```
18
31
45
70
82
```

Por que usar dados sintéticos?

- praticar sem dados pessoais
- repetir testes rapidamente
- criar casos variados
- separar programação de interpretação clínica

Funções: dar nome a uma tarefa

Uma função recebe dados, executa uma tarefa e pode devolver um resultado.



Vantagens: clareza · reuso · teste isolado

Nossa função classificar(valor)

Extraímos a decisão para um bloco reutilizável.

FUNÇÃO + TESTES

```
def classificar(valor):
    if valor < 30:
        return "A"
    elif valor <= 70:
        return "B"
    return "C"

print(classificar(29))
print(classificar(30))
print(classificar(71))
```

SAÍDA

```
A
B
C
```

DEF

define a função

VALOR

parâmetro de entrada

RETURN

devolve o resultado

return não é a mesma coisa que print

Mostrar um valor na tela é diferente de devolvê-lo para o restante do programa.

`print()`

mostra informação para a pessoa

```
def mostrar(valor):
    print(valor)

resultado = mostrar(52)
```

`return`

entrega um valor ao programa

```
def devolver(valor):
    return valor

resultado = devolver(52)
```

Para montar registros, precisamos do valor retornado pela função.

Dicionários: um registro com campos nomeados

Eles ajudam a representar uma leitura como um conjunto de informações relacionadas.

UM REGISTRO

```
registro = {
    "valor_sensor": 52,
    "estado_didatico": "B",
    "qualidade": "OK"
}

print(registro["estado_didatico"])
```

chave → valor

```
"valor_sensor" → 52
"estado_didatico" → "B"
"qualidade" → "OK"
```

SAÍDA

```
B
```

De leituras a registros

Lista + função + dicionário formam o núcleo do projeto final.

MONTANDO UMA LISTA DE REGISTROS

```
leituras = [18, 31, 45, 70, 82]
registros = []

for valor in leituras:
    registro = {
        "valor_sensor": valor,
        "estado_didatico": classificar(valor),
        "qualidade": "OK"
    }
    registros.append(registro)
```

Fluxo



DESAFIO INTEGRADO · 15 MIN

Construa o monitor didático

A partir de uma lista de leituras sintéticas, gere uma lista de registros com valor, faixa didática e qualidade técnica.

lista | função | for | dicionário | testes

Critério mínimo: o notebook executa do início ao fim e passa nos casos 29, 30, 70 e 71.

Estrutura sugerida

Complete as partes marcadas sem copiar a solução inteira.

ESQUELETO

```
leituras = [18, 31, 45, 70, 82]

def classificar(valor):
    # TODO: retornar A, B ou C
    ...

registros = []
for valor in leituras:
    registros.append({
        "valor_sensor": valor,
        "estado_didatico": classificar(valor),
        "qualidade": "OK"
    })

print(registros)
```

Checklist

- a função retorna um valor
- o for processa todas as leituras
- cada item é um dicionário
- os quatro casos de fronteira passam
- você consegue explicar o fluxo

Terminou antes? Aumente o desafio

As extensões são opcionais — primeiro valide a solução mínima.

1 · CONTADORES

conte quantas leituras caem em A, B e C

2 · RANDOM

gere uma nova lista de leituras sintéticas

3 · IDENTIFICADOR

adicione id_dispositivo a cada registro

4 · VALIDAÇÃO

recuse valores fora de 0–100

Regra: extensão só depois de a versão básica funcionar e os testes de fronteira passarem.

Depurar é investigar

Quando o resultado não é o esperado, procure o erro de forma sistemática.

Tipo errado	**input() ainda é str**	→ use type() e converta
Indentação	**bloco saiu do if/for**	→ compare o alinhamento
Condição	**= em vez de == ou ordem incorreta**	→ teste 29/30/70/71
Loop	**só parte da lógica repete**	→ coloque o fluxo completo dentro do laço

Pergunte sempre: qual valor está na variável? qual condição está sendo testada? quantas vezes esta linha executa?

Do Python aos próximos cursos

Você já tem o núcleo que será reutilizado na trilha.

Python básico

variáveis + regras

Coleções

listas + dicionários

PR-03

Pandas + tabelas

PR-04

dados em tempo real

QUIZ DE SAÍDA

Explique em 3 frases: qual é a entrada, qual é o processamento e qual é a saída do seu programa?

Você não precisa memorizar tudo. Precisa saber reconhecer, testar e adaptar.